

MASTER PRODUCT REQUIREMENTS DOCUMENT

AI Job Application Copilot

Version 2.1 — Final Master Implementation Specification

Status: Master implementation specification

Date: September 12, 2026

Environment: Browser extension + backend + AI fallback

Core principle: Deterministic first, AI only when necessary

v2.1 changes: Adds an explicit resume-parser failure → manual-entry fallback and clarifies extension UI-stack rules. It also formalizes sensitive-question safety, emergency stop, privacy/data handling, adapter health/versioning, and production safeguards while keeping detailed UX and context-ranking decisions implementation-level.

1. Executive Summary

Build a browser extension that assists users with job applications across Naukri, LinkedIn, Indeed, and supported ATS platforms. It must handle both traditional forms and conversational/chatbot applications. The system automates repetitive work while keeping factual control and final submission with the user during the MVP. It must stop safely on CAPTCHA, human verification, login requirements, unsupported UI, or unresolved uncertainty.

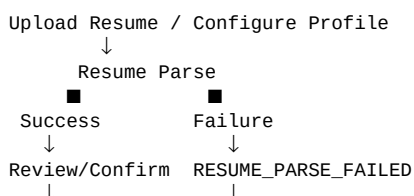
2. Product Goals

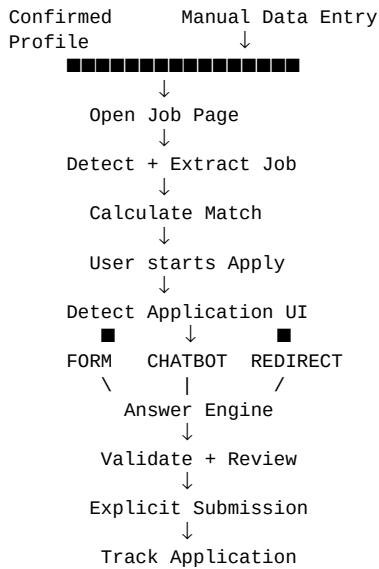
- Detect job pages and extract useful job information.
- Maintain a structured, user-confirmed professional profile.
- Calculate an explainable job/profile match score.
- Detect form, chatbot, external redirect, or unknown application UI.
- Fill traditional forms using deterministic profile mapping.
- Handle one-question-at-a-time chatbot applications.
- Use AI only as controlled fallback for complex/free-text/unknown questions.
- Never fabricate user facts.
- Allow review, editing, pausing, resuming, cancellation, manual takeover, and emergency stop.
- Track applications and detailed events.
- Support site-specific adapters without coupling generic logic to a specific website.

3. Non-Goals / Explicit Boundaries

- Do not bypass CAPTCHA, anti-bot controls, authentication protections, or other security mechanisms.
- Do not fabricate qualifications.
- Do not assume every site exposes a stable API or private endpoint.
- Do not build all sites simultaneously during MVP.
- Do not make final submission fully autonomous in the initial version.
- Do not send the full resume/profile to AI when smaller relevant context is sufficient.

4. Core User Journey





5. Critical Application Modes

5.1 Form Mode

Traditional pages may contain text inputs, textareas, dropdowns, radio buttons, checkboxes, uploads, and custom controls.

5.2 Chatbot Mode

A conversational application asks a question, receives an answer through text or selection controls, submits it, and presents another question. Repeat until completion or a safe-stop condition.

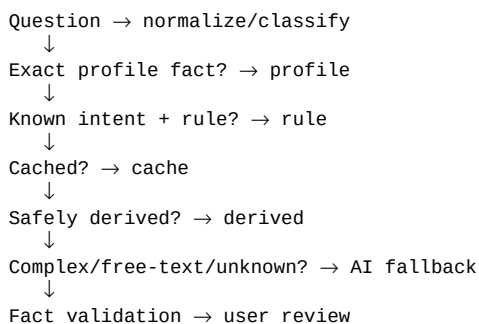
5.3 External Redirect

Detect the destination and hand off to a supported adapter.

5.4 Unknown

Pause and request user intervention rather than guessing.

6. Hybrid Answer Architecture



AI is a fallback, not the primary application driver.

7. User Profile — Source of Truth

- Personal: full name, email, phone, location.
- Professional: current role, total experience, skills, skill experience, employers, job titles, projects.
- Education: degree, institution, graduation year.
- Preferences: preferred locations, remote preference, relocation preference, notice period, expected salary, current salary.
- Additional: certifications, languages, and work authorization when explicitly configured by the user.

Profile data must be user-confirmed. AI may normalize/classify but must not silently alter confirmed facts.

8. Resume Processing

- Accept PDF and DOCX.
- Extract text and parse structured candidate information.
- Display extracted information and allow corrections.
- Persist only confirmed information as authoritative profile data.

```

Upload → Parse
  ■■ Success → extract → review/correct → confirm
  ■■ Failure → RESUME_PARSE_FAILED
               → explain failure
               → manual data entry
               → confirm → continue

```

Parsing uses bounded attempts. Parser failure must never cause an infinite retry loop. Manual data becomes authoritative only after user confirmation.

9. Job Extraction

- Job title
- Company
- Location
- Description
- Required experience
- Skills
- Salary when available
- Job URL
- Source
- External application URL when available

Adapters own platform-specific extraction. Generic components must not contain platform-specific selectors.

10. Job Matching

Generate an explainable score across skills, experience, education, and location. Prefer deterministic matching; AI may assist difficult parsing or normalization. Never use the score to justify falsifying answers.

11. Application State Machine

```

IDLE → JOB_DETECTED → APPLICATION_STARTED → DETECTING_UI
  ■→ FORM_MODE
  ■→ CHATBOT_MODE
  ■→ EXTERNAL_REDIRECT
  ■→ UNKNOWN_UI
  ↓
READING_QUESTION / DETECTING_FIELDS
  ↓
CLASSIFYING → GENERATING_OR_RETRIEVING_ANSWER
  ↓
VALIDATING → WAITING_FOR_USER_REVIEW (when required)
  ↓
FILLING / SELECTING → SUBMITTING → WAITING_FOR_NEXT_STATE
  ↓
QUESTION_RECEIVED → repeat

Terminal/interruption:
APPLICATION_COMPLETED, ALREADY_APPLIED, CAPTCHA_DETECTED,
LOGIN_REQUIRED, USER_PAUSED, USER_CANCELLED, APPLICATION_FAILED,
TIMEOUT, UNSUPPORTED_UI, RESUME_PARSE_FAILED

```

12. Form Engine

- Detect inputs, textareas, selects, checkboxes, radios, uploads, buttons, and custom controls.
- Map using labels, placeholder, aria-label, name, autocomplete, nearby text, DOM context, and adapter rules.
- Use deterministic profile mapping before AI.

- Validate values before submission.
- Record every filled field and answer source.

13. Chatbot Engine

- Detect conversation containers, bot/user messages, inputs, options, and send/submit controls.
- Extract only new bot messages; never treat the extension's own answer as a new question.
- Support one or multiple questions per bot message.
- Classify intent and answer from profile/rules/cache first.
- Use AI only when deterministic processing is insufficient.
- Validate generated answers against confirmed facts.
- Type/select, submit, and observe the next state.
- Detect completion, errors, CAPTCHA, login, timeout, and unsupported states.
- Support pause, resume, cancel, manual takeover, and emergency stop.

14. Chatbot Message Model

```
Message { id: string; role: "BOT" | "USER" | "SYSTEM"; text: string;
  timestamp?: string; domFingerprint?: string; }
```

```
Question { messageId: string; text: string; intent: QuestionIntent;
  interactionType: InputType; confidence: number; }
```

15. Question Intent Catalog

NAME, EMAIL, PHONE, LOCATION, TOTAL_EXPERIENCE; SKILL_BOOLEAN, SKILL_EXPERIENCE, SKILL_LIST; EDUCATION, DEGREE, CERTIFICATION; SALARY_CURRENT, SALARY_EXPECTED; NOTICE_PERIOD, RELOCATION, REMOTE_WORK, WORK_AUTHORIZATION, AVAILABILITY; FREE_TEXT, UNKNOWN.

16. Multiple-Question Handling

Segment multiple questions into independent questions and resolve separately. If the UI expects one combined response, preserve its expected format. If ambiguous, ask the user.

17. Question Cache & Duplicate Protection

- Normalize question text.
- Maintain question fingerprints and processed-message IDs.
- Cache reusable intent classifications and safe reusable answers where appropriate.
- Do not process the same DOM message twice after re-render.
- Do not submit the same answer twice unless explicitly requested again.
- Persist application state so refresh/re-render cannot cause accidental duplicate submissions.

18. Answer Sources

Every answer records PROFILE, RULE, CACHE, DERIVED, AI, or USER as its source.

19. AI Fallback Contract

```
Request:
{ "question": "...", "relevant_profile": {},
  "relevant_job_context": {}, "relevant_conversation_context": [] }
```

```
Response:
{ "answer": "...", "confidence": 0.0,
  "requires_user_confirmation": true, "reason": null }
```

Send only minimum relevant confirmed context. The first implementation may use a straightforward relevance strategy; context selection can be optimized through testing.

20. AI Fact Safety

- Never invent experience, skills, employers, degrees, certifications, salary, visa/work authorization, employment history, or other personal facts.
- Missing required information → requires_user_confirmation=true.
- Low-confidence answers require review.
- AI cannot change confirmed profile facts.
- Validate AI output before insertion.

21. Sensitive / High-Risk Answer Policy

Do not auto-answer work authorization/visa status, disability or accommodation information, criminal/legal questions, demographic information, salary/CTC without an explicit configured value, or relocation without an explicit configured preference. Pause and ask the user.

22. AI Cost & Latency Controls

- Prefer local/profile/rule answers.
- Cache intent classifications and safe reusable answers.
- Send only relevant profile/job context; never resend the full resume for every question.
- Track calls, tokens when available, latency, and failures.
- Allow per-application AI call/token limits.
- If the budget is exhausted, pause and ask the user.

23. User Override, Pause, Resume & Emergency Stop

- User can edit or replace any proposed answer.
- User can pause, resume, cancel, or take manual control.
- Provide a persistent emergency STOP control that immediately halts automation actions for the active application.

24. Review Interface

The MVP must implement WAITING_FOR_USER_REVIEW with the proposed answer, source, confidence when available, and appropriate approve/edit/replace/skip/pause/cancel controls. Exact visual polish is implementation-level.

25. Completion & Safe Stop

- Detect successful submission, already-applied, failure, login-required, CAPTCHA/human verification, unsupported UI, and timeout.
- Stop safely whenever state is uncertain.
- Never continue clicking blindly after an unexpected state.

26. Synchronization & Mutation Handling

Use MutationObserver and equivalent event-driven mechanisms. Fixed delays are secondary safeguards only.

27. Timeout & Retry Policy

- After submit, wait for meaningful state/message change.
- If none occurs, re-check state before retrying.
- Never repeatedly click submit without evidence the prior action failed.
- Use bounded retries.
- After retry limit, transition to TIMEOUT or APPLICATION_FAILED and ask the user.

28. CAPTCHA & Security

Never bypass CAPTCHA, human verification, anti-bot systems, authentication protections, or other security controls. Pause for manual user action and resume only from a safely recognizable state.

29. Site Adapter Architecture

```
BaseAdapter
■ ■ NaukriAdapter ← MVP priority
■ ■ LinkedInAdapter
■ ■ IndeedAdapter
■ ■ GreenhouseAdapter
■ ■ LeverAdapter
■ ■ WorkdayAdapter
■ ■ AshbyAdapter
■ ■ GenericAdapter
```

Adapters may implement extraction, application detection, DOM-specific interaction, and completion detection. Generic engines remain platform-independent.

30. Naukri MVP Priority

Naukri is first. MVP must validate both Naukri form-style and chatbot-style flows before expansion. Never assume current selectors, APIs, endpoints, or DOM structure; inspect actual test behavior first.

31. Generic ATS Strategy

After Naukri is stable, add reusable ATS adapters rather than company-specific implementations. Generic engines handle common interaction patterns; adapters handle platform-specific behavior.

32. Browser Extension Architecture & UI Stack

```
extension/
  src/background/service-worker.ts
  src/content/content.ts
  src/content/application-agent.ts
  src/content/form-detector.ts
  src/content/chatbot-detector.ts
  src/content/message-parser.ts
  src/content/question-classifier.ts
  src/content/interaction-engine.ts
  src/content/state-machine.ts
  src/adapters/{base,naukri,linkedin,indeed,greenhouse,lever,workday,ashby,generic}.ts
  src/components/{JobCard,MatchScore,ApplyPanel,ReviewPanel,ProfilePanel}.tsx
  src/popup/App.tsx
  src/utils/{api,storage}.ts
```

UI stack rule: Preserve the existing repository's suitable frontend framework, build system, TypeScript configuration, and styling approach. Do not replace the stack simply because another framework is preferred. If a new UI stack is necessary, Phase 0 must identify options and OpenCode must ask the developer to approve the choice before implementation. Extension UI styles must be isolated from host-page CSS.

33. Backend Architecture

```
backend/app/
  main.py
  api/{auth,profile,resume,jobs,questions,applications,analytics}.py
  services/{resume_parser,job_extractor,job_matcher,
            question_classifier,answer_engine,ai_service,
            application_service}.py
  models/  schemas/  database/  tests/
```

34. API Contract — Initial

POST /api/profile; GET /api/profile; PATCH /api/profile; POST /api/resumes; POST /api/jobs/analyze; POST /api/jobs/match; POST /api/questions/classify; POST /api/questions/answer; POST /api/applications; GET /api/applications; GET/PATCH /api/applications/{id}; POST/GET /api/applications/{id}/events; GET /api/usage/ai.

OpenCode may refine contracts after repository inspection, but must document changes and ask for clarification when architecture is materially affected.

35. Database Model

users, profiles, skills, experience, education, preferences, resumes, jobs, job_matches, applications, application_answers, application_events, question_intents, question_cache, ai_usage.

35.1 Application Answer Record

id, application_id, question, intent, interaction_type, answer, answer_source, confidence, was_ai_generated, user_modified, created_at.

35.2 Application Event Record

id, application_id, event_type, state, message, metadata, created_at.

36. Application Events

APPLICATION_STARTED; APPLICATION_TYPE_DETECTED; QUESTION_RECEIVED; QUESTION_CLASSIFIED; ANSWER_RETRIEVED; AI_REQUESTED; ANSWER_GENERATED; ANSWER_MODIFIED; ANSWER_SUBMITTED; USER_PAUSED; USER_RESUMED; CAPTCHA_DETECTED; LOGIN_REQUIRED; APPLICATION_COMPLETED; APPLICATION_FAILED; TIMEOUT; RESUME_PARSE_FAILED; EMERGENCY_STOP.

37. Application Tracker

Company, Job title, URL, Source, Match score, Resume used, Application date, Status, History. Statuses: SAVED, READY, IN_PROGRESS, REVIEW_REQUIRED, APPLIED, INTERVIEW, REJECTED, WITHDRAWN, FAILED.

38. Security & Environment Variables

```
APP_ENV=development
API_BASE_URL=
DATABASE_URL=
JWT_SECRET=
AI_PROVIDER=
AI_API_KEY=
AI_MODEL=
STORAGE_PROVIDER=
STORAGE_BUCKET=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
SENTRY_DSN=
```

Never hard-code secrets. Maintain .env.example and keep .env out of version control. Missing required configuration → stop and ask for exact variable, purpose, and provider. Never guess or expose credentials.

39. Privacy, Permissions & Data Handling

- Request only minimum browser permissions needed.
- Do not collect/transmit unrelated page content.
- Separate data stored locally, sent to backend, and sent to AI provider.
- Send only minimum relevant context to AI.
- Protect profile, resume, application, and auth data in transit and at rest according to the chosen architecture.
- Define retention/deletion for resumes, profiles, answers, events, and AI usage before production.
- Do not log secrets, auth tokens, or unnecessary personal data.

40. OpenCode Clarification Protocol

- Missing API key/configuration.
- AI provider/model not selected.
- Database/storage/auth choice ambiguous.
- Website DOM behavior cannot be reliably determined.
- New interaction pattern is outside adapter contract.
- Platform technical or contractual restrictions unclear.
- Feature conflicts with safety/security.
- Materially different architectural choices exist.

OpenCode must not hide uncertainty by inventing an implementation.

41. OpenCode Secret Prompt Format

Required configuration is missing.
Variable: AI_API_KEY
Purpose: Generate answers for unresolved application questions.
Provider: <selected AI provider>
Please provide the value or confirm the provider/configuration to use.
Do not continue with a fabricated credential.

42. OpenCode Engineering Rules

- Inspect the existing repository completely before changing anything.
- Do not rewrite functionality without understanding it.
- Implement incrementally.
- Run tests, type checking, linting, and builds after each major phase.
- Do not invent APIs, credentials, selectors, endpoints, or external-service behavior.
- Keep website-specific logic inside adapters.
- Use event-driven observation.
- Never bypass security controls.
- Require explicit user confirmation before final submission.
- Document assumptions and limitations.
- Ask the developer when required information is missing.
- Do not claim adapter reliability without representative testing.
- Do not silently replace the existing frontend stack.

43. Development Phases

Phase 0 — Repository Analysis

Inspect the existing project. Produce architecture report, dependency inventory, risk list, current functionality map, required configuration list, frontend/UI stack assessment, and implementation plan. Do not implement product functionality.

Phase 1 — Foundation

Extension/backend foundations, environment configuration, database foundation, authentication foundation, documentation.

Phase 2 — Profile

Structured profile, preferences, persistence, validation, editing.

Phase 3 — Resume

PDF/DOCX extraction, parsing, review, correction, confirmed profile generation, bounded retries, RESUME_PARSE_FAILED, and manual-entry fallback.

Phase 4 — Job Detection + Naukri

Job extraction and Naukri adapter; validate actual page behavior first.

Phase 5 — Matching

Explainable deterministic matching and optional AI normalization.

Phase 6 — Form Engine

Form detection, control classification, profile mapping, autofill, validation, audit records, review UI.

Phase 7 — Chatbot Engine

State machine, chatbot detection, message extraction, segmentation, intent classification, deterministic answers, duplicate protection, MutationObserver, interaction controls, pause/resume, emergency stop, timeout/retry, completion detection, safe stops.

Phase 8 — AI Fallback

Provider abstraction, minimal-context prompting, structured responses, fact validation, confidence, clarification, sensitive-question policy, cost controls, caching, usage tracking.

Phase 9 — Tracker

Application records, statuses, history, events, analytics.

Phase 10 — Additional Adapters

LinkedIn, Indeed, and reusable ATS adapters after Naukri stability.

Phase 11 — Reliability

Fixtures, integration/regression tests, security/privacy tests, performance tests, and failure recovery.

44. Testing Strategy

44.1 Unit Tests

- Question normalization
- Intent classification
- Profile mapping
- Answer validation
- State transitions
- Duplicate fingerprints
- AI response validation
- Cost-limit enforcement
- Resume parser failure transition
- Sensitive-question policy

44.2 Chatbot Fixtures

- Q → answer → next Q → answer → complete.
- Unknown Q → clarification → continue.
- Complex Q → AI → validated answer → continue.
- Yes/No controls.
- Dropdown/radio/button controls.
- Free-text.
- Multiple questions in one message.
- DOM re-render without duplicate processing.
- Duplicate bot message.
- Submit timeout.
- CAPTCHA.
- Login-required.
- Already-applied.
- Unexpected UI → safe stop.
- Emergency STOP → no further automation actions.

44.3 Integration Tests

Test extension/backend communication, profile persistence, resume fallback, job analysis, question processing, state transitions, event logging, and AI fallback.

44.4 Adapter Tests

Each adapter must have representative fixtures and regression tests. Platform DOM changes should be isolated to the adapter unless the generic contract is genuinely insufficient.

45. Metrics & Observability

- Applications started/completed/failed
- Questions processed
- Deterministic/AI/user answers
- User-modified answers
- Unknown questions
- AI calls per application
- AI tokens/cost when available
- AI latency
- Timeouts
- CAPTCHA/manual-intervention events
- Adapter failures
- Resume parser failures/fallback usage
- Emergency-stop events

Example metric values, if shown in documentation, are illustrative only.

46. Adapter Health & Versioning

Adapters should expose a version and health status, e.g. Naukri Adapter v1.0 — Healthy — last validated . If regression tests fail after a platform change, mark the adapter unhealthy and stop/restrict automation rather than silently continuing.

47. MVP Definition of Done

- Chrome extension operates reliably in the supported test environment.
- User can create/edit structured profile.
- User can upload and confirm resume-derived profile.
- Parser failure safely reaches manual data entry.
- Naukri job information detected in supported flows.
- Explainable job match.
- Naukri form flow works in supported tests.
- Naukri chatbot processes one question at a time and supports text, Yes/No, buttons, radio, and dropdown where present.
- Deterministic answers require no AI.
- AI is fallback only and validated against confirmed facts.
- Sensitive/high-risk questions are not auto-answered without explicit configured data.
- User can edit, pause, resume, cancel, and emergency-stop.
- Duplicate messages do not duplicate submissions.
- CAPTCHA causes safe pause.
- Final submission requires explicit confirmation.
- Events and answer sources are recorded.
- Critical chatbot/parser tests pass.
- No secrets are hard-coded or committed.

48. Future Expansion

- LinkedIn
- Indeed
- Greenhouse/Lever/Workday/Ashby
- More robust generic ATS detection
- Improved semantic classification
- Personalized answer templates
- Application analytics
- Interview/outcome analytics
- Resume-version tracking
- Dry-run/test mode that detects/previews actions without typing or submitting

49. Master OpenCode Execution Instruction

OpenCode must treat this document as the master product specification.

START WITH PHASE 0 ONLY.

1. Inspect repository completely.
2. Identify architecture and reusable components.
3. Identify dependencies and build/test systems.
4. Identify missing configuration and secrets.
5. Identify technical ambiguities.
6. Identify risks and platform-specific uncertainties.
7. Assess the existing frontend/UI stack against this PRD.
8. Produce ARCHITECTURE.md and an implementation plan.
9. Do not implement Phase 1 until Phase 0 is complete and the developer has reviewed/approved the plan.

For every subsequent phase: explain changes; implement only the approved phase; run tests, type checking, linting, and builds; fix errors; report files changed, configuration, limitations, test/build results, and next phase.

If information is missing: STOP and ask the developer.

Never: invent credentials, selectors, APIs, user facts; bypass security controls; silently change architecture; or claim unsupported platform behavior is reliable.

50. Final Product Principle

The product should feel like an intelligent assistant, not an uncontrolled bot. Deterministic profile data handles predictable questions; AI handles only the difficult remainder; the state machine controls the workflow; adapters isolate website differences; user controls provide safe intervention; and detailed events make every application action observable and debuggable.